PROJECT PROPOSAL

GameTrainer

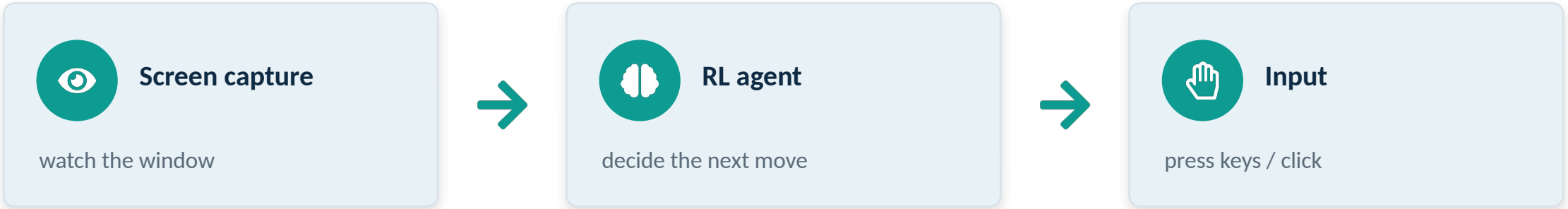
A local, vision-based AI that learns to play games — built around a clean, swappable architecture.

Proposal · **crawl-first plan, M0 → M6**

THE PROPOSAL

What I want to build

An agent that learns to play a game from **pixels** — it watches the screen like a person, decides a move, and sends **keyboard / mouse input** back. No memory reading, no touching the game's code.



First target after the warm-ups: Stardew Valley — but the engine is meant to be game-agnostic.

100% local · $0 API cost · runs on my own hardware

WHY THIS IS INTERESTING

The point isn't the AI — it's the architecture

One fixed contract in the middle. Any game and any AI brain plug into it without knowing about each other. That swappability is the whole thesis.

Ground

The game world

Shows what's happening + grades the AI.

AI

The player

Looks, decides, acts.

Link

The Gymnasium API

The socket both plug into.

The loop, forever:

`observe` → `act` → `reward` → `repeat`

Design pattern: Ports & Adapters (hexagonal architecture).

SMART SCOPING

Build what's mine; borrow what's solved

I don't reinvent the learning algorithm or the vision model. I build the parts that make it swappable.

I BUILD

The Ground
the game worlds and their reward rules

The Link
the socket + per-game profiles (config, not code)

I BORROW

The Brain
PPO from Stable-Baselines3 — proven RL

The Eyes backbone
a Vision Transformer pretrained on ImageNet

Net effect: the hard, novel work is the architecture — exactly where the learning value is.

SCOPE DISCIPLINE

In scope vs. deliberately not (yet)

In scope (v1)

- Pixels in, actions out — perceive like a human
- Crawl-first: CartPole → GridWorld → vision → real game
- Swappable perception (numeric → ViT) and input
- Per-game “profiles” for config
- Runs locally on CPU early, GPU later

Not doing (on purpose, for now)

- No memory reading or process injection
- No C++ in v1 — Python input is fast enough
- No control discovery — controls live in a profile
- No cloud or paid APIs
- Don't start on Stardew — earn the way up

THE PLAN

Crawl-first roadmap — M0 to M6

Each step proves one new idea before the next. Finishing M4 already proves the whole thesis.

M0	**Setup**	CartPole runs on random actions	✓ *validated*
M1	**Borrow the brain**	PPO learns CartPole through my runner	✓ *validated*
M2	**Build my own Ground**	A GridWorld that obeys the contract	
M3	**Add the Eyes**	ViT learns from a picture, not numbers	
M4	**Make it swappable**	Switch games by config only — no code	**thesis proven here**
M5	**Add the Hands**	Real key presses drive a live game	
M6	**Stretch: Stardew**	Stardew becomes “just another profile”	

WHAT SUCCESS LOOKS LIKE

Clear, measurable “Done when…” at every step

Primary success = Milestone 4

Switching between games is config-only, with zero code edits. That proves it: any ground, any brain, one socket.

Every milestone is measurable

Each has a concrete “Done when…” check before moving on.

The contract never breaks

reset() / step() stays identical across every game.

Portfolio win at M4

M5 (real hands) and M6 (Stardew) are bonus, not requirements.

HONEST NOTES

Risks — and how I'll handle them

Reward from pixels is brittle

Reading a score off-screen is error-prone → start with simple worlds, save Stardew for last.

Breaking the contract kills the point

If I bend reset()/step() to “make it work,” swappability is gone → treat the contract as untouchable.

Light hours each week

Big unfinished steps stall → smallest next brick only; never scaffold future phases early.

AMD-on-Windows GPU is fiddly

ROCm/DirectML can fight back → run on CPU until the ViT (M3); sort the GPU before then.

The ask

1

Green-light the crawl-first plan

Back the M0 → M6 sequence. The discipline of building in order is the plan, not a constraint.

2

Judge it at M4, not M6

Success is config-only game-swapping. Stardew is a stretch, not the bar.

3

First two steps are already de-risked

M0 and M1 are built and validated — see the milestone reviews.